

Sistema Automatizado de Gestión de Estacionamiento

ParkScan



Integrantes

- Anzaldo Kevin
- Atencio Juan Manuel
- Muñoz Maximo
- Navas Francisco
- Penoff Pablo

Etapa 1

ETAPA 1: DEFINICIÓN DEL PROYECTO Y CASO DE NEGOCIO

1.1 Presentación y Justificación del Proyecto

El proyecto consiste en el diseño y desarrollo de ParkScan, un sistema automatizado de gestión integral para playas de estacionamiento. El núcleo del sistema está diseñado para transformar la gestión operativa mediante cuatro pilares fundamentales: control de flujo de vehículos, optimización de activos a través del monitoreo en tiempo real, gestión transaccional segura y generación de inteligencia de negocio mediante reportes detallados.

Justificación Técnica y de Negocio: La propuesta surge como respuesta a la necesidad de modernizar estacionamientos de distintas dimensiones que actualmente operan de forma manual. El conteo manual suele generar errores críticos, demoras operativas y falta de transparencia financiera. ParkScan se diferencia por su enfoque de Producto Mínimo Viable (MVP): ofrece una alternativa económica y escalable que reduce costos de infraestructura al no depender de sensores físicos complejos, utilizando en su lugar un modelo lógico basado en tecnología de códigos QR para el registro de estadías.

1.2 Análisis del Entorno

Entorno Interno (La Organización)

La organización objetivo es una empresa de gestión de estacionamientos con una estructura operativa pequeña y un grado de digitalización bajo, donde predominan los registros en papel o herramientas informáticas aisladas. A pesar de esto, existe una visión estratégica hacia la modernización y la optimización de procesos para reducir el error humano. El equipo de desarrollo cuenta con los conocimientos técnicos en programación (Python/Flask) y bases de datos (MySQL) necesarios para ejecutar el proyecto, apoyándose en software libre para garantizar la viabilidad.

Entorno Externo (El Mercado)

El mercado de estacionamientos urbanos en zonas céntricas (como Mendoza o San Rafael) es altamente competitivo y está regulado por normativas municipales sobre tarifas y seguridad. Si bien existen sistemas de alta gama con barreras automatizadas y sensores de masa, su elevado costo de implementación los vuelve inaccesibles para comercios medianos. **ParkScan** ocupa este nicho, proporcionando una herramienta profesional que proyecta una imagen corporativa adecuada y cumple con los requisitos de licenciamiento abierto.

1.3 Objetivos y Alcances Preliminares

Objetivos Estratégicos

- **Control de Ocupación:** Gestionar en tiempo real la disponibilidad de espacios por sectores y de forma global para optimizar el uso del predio.
- **Automatización Transaccional:** Implementar la emisión y validación de tickets QR para agilizar el ingreso, el cálculo exacto de tarifas y el proceso de cobro.
- **Trazabilidad y Auditoría:** Generar una base de datos confiable que permita auditar la recaudación diaria, segregada por turnos y operarios responsables.
- **Experiencia del Usuario:** Ofrecer una interfaz minimalista y funcional que simplifique la interacción tanto para el personal de playa como para los conductores.

Alcance del Software

El sistema **ParkScan** integrará las siguientes capacidades funcionales:

- **Arquitectura de Tres Capas (Three-Tier Architecture):** El sistema se estructurará mediante una separación modular estricta para garantizar la escalabilidad y el mantenimiento.
- **Capa de Presentación (Frontend):** Interfaces de usuario diferenciadas para Administradores, Operarios y Clientes.
- **Capa de Lógica de Negocio (Backend):** Desarrollada en **Flask (Python)**, encargada de orquestar la lógica transaccional, el cálculo de tarifas y la comunicación con servicios externos.
- **Capa de Persistencia (Datos):** Una base de datos relacional en **MySQL** que asegura la integridad de los registros operativos y financieros.

Gestión de Inventario y Activos: Módulos específicos para la administración granular de la topología física del estacionamiento.

- **Sectores:** Segmentación del predio para una mejor organización logística.
- **Plazas:** Identificación individual y monitoreo en tiempo real del estado de cada espacio (Libre/Ocupado).

Módulo de Gestión Transaccional y Pasarela de Pagos (API): Este componente es el núcleo financiero del sistema y cumple con el requisito de integración externa.

- **Integración de API de Pagos:** Consumo de servicios externos (como Mercado Pago u otras pasarelas electrónicas) para administrar cobros de forma segura y automatizada.
- **Cálculo Dinámico de Tarifas:** Algoritmo que procesa el tiempo exacto de estadía para generar el monto a abonar, eliminando errores de cálculo manual.
- **Validación de Salida:** Sistema que confirma la conciliación automática del pago para habilitar la liberación de la barrera de salida.

Panel Administrativo e Inteligencia de Negocio: Interfaz estratégica para la toma de decisiones basada en datos reales.

- **Métricas de Desempeño:** Visualización en tiempo real de la ocupación, flujo de ingresos y volumen de tráfico.
- **Reportes Operativos:** Generación de informes exportables en formatos normalizados como **PDF y CSV**.
- **Auditoría de Cierres Contables:** Mecanismo de consolidación financiera diaria que agrupa transacciones por turno y operario para garantizar la transparencia total.

Etapa 2

ETAPA 2: ANÁLISIS DE FACTIBILIDAD Y PROPUESTA TÉCNICA

2.1 Matriz de Factibilidad Multidimensional

Para asegurar que el proyecto sea ejecutable y sostenible, se ha realizado un desglose exhaustivo de los factores de viabilidad:

- **Factibilidad Técnica:** El proyecto es técnicamente posible ya que se apoya en tecnologías conocidas y accesibles para el equipo, como Python y Flask para la lógica de backend, y MySQL para la persistencia de datos. El equipo cuenta con los conocimientos necesarios adquiridos en el cursado para manejar la concurrencia de usuarios y la conexión a bases de datos. Además, al no requerir integración con software externo complejo inicialmente, se reducen los riesgos técnicos significativamente. Los riesgos identificados, como el manejo de concurrencia y la integridad de los datos, son manejables mediante la aplicación de buenas prácticas de desarrollo y pruebas de estrés.
- **Factibilidad Económica:** El proyecto es económicamente viable, ya que no requiere inversión en hardware especializado. Todo el desarrollo se realizará con software libre o gratuito, como Python, Flask, MySQL y Figma. La inversión principal reside en el tiempo de trabajo del equipo. A largo plazo, el costo-beneficio es positivo, ya que la automatización optimiza la gestión, evita errores humanos y reduce costos operativos en estacionamientos.
- **Factibilidad Operativa:** El proyecto puede llevarse a cabo sin inconvenientes operativos, dado que el equipo posee competencias en programación, diseño de bases de datos y gestión ágil. El sistema no modifica flujos de trabajo complejos; simplifica las tareas de control reemplazando el conteo manual por una interfaz automatizada. La adopción por parte del personal será fluida debido a una interfaz diseñada bajo principios de simplicidad.
- **Factibilidad Legal:** El sistema cumple con la normativa legal vigente, ya que no recopila información sensible de los usuarios finales. Los datos manejados (patente, hora de entrada/salida, sector) son de carácter operativo. Asimismo, se utilizan librerías con licencias abiertas como ZXing y PyQRCode, cumpliendo con los requisitos de licenciamiento de software libre.

- **Factibilidad Política:** Actualmente, el proyecto no cuenta con el apoyo de Stakeholders externos (interesados reales), por lo que se define como una iniciativa interna de carácter académico.

2.2 Metodología de Desarrollo y Ciclo de Vida del Software

Se ha seleccionado el marco de trabajo Scrum, alineado con un enfoque ágil que permite la entrega de valor incremental:

- **Desarrollo Iterativo:** El trabajo se divide en sprints cortos para entregar avances funcionales constantes, permitiendo una mejor adaptación a cambios y retroalimentación continua.
- **Marco PMBOK®:** La gestión integral se rige bajo los lineamientos del PMBOK® 7ma Edición, asegurando que el sistema evolucione de forma consistente hasta su despliegue final.
- **Justificación Académica:** El enfoque ágil se ajusta de manera óptima a la dinámica de trabajo y los tiempos del cuatrimestre universitario.

2.3 Propuesta Técnica Detallada (Stack Tecnológico)

La infraestructura tecnológica se ha definido priorizando la robustez, el mantenimiento y la escalabilidad:

- **Backend y Lógica:** Empleo de Python con el framework Flask, priorizando la robustez y la facilidad de mantenimiento.
- **Persistencia de Datos:** Uso de MySQL normalizado para asegurar la persistencia, administración de transacciones y auditoría.
- **Tecnología QR:** Integración de bibliotecas ZXing y PyQRCode para la generación y escaneo de códigos bidimensionales.
- **Reportería:** Generación de informes operacionales en formatos normalizados como PDF y CSV.
- **Integraciones (API):** Se prevé la integración de pasarelas de pago electrónico para administrar cobros, agilizar el flujo de salida y conciliar automáticamente el estado del ticket.

2.4 Diseño de Interfaz y Arquitectura de Información

El diseño se fundamenta en el minimalismo funcional, dividiendo la arquitectura en cuatro componentes esenciales:

1. Emisión: Generación rápida de tickets al ingreso del vehículo.
2. Administración: Visualización en tiempo real de la ocupación por sectores.
3. Caja: Escaneo, cálculo de tarifas y procesamiento de pagos.
4. Validación: Control final de barrera para garantizar el egreso seguro.

2.5 Prototipo UI/UX (Mockups)

El prototipo visual refleja la simplicidad operativa necesaria para el flujo constante de una playa de estacionamiento:

- Interfaz de Entrada: Pantalla de bienvenida con opción clara de "ENTRADA".
- Ticket Digital: Comprobante que incluye el ID del ticket, la hora de entrada exacta y la recomendación de conservarlo para la salida.
- Interfaz de Salida: Pantalla de despedida ("¡Hasta Pronto!") que confirma la finalización del servicio tras el pago.

2.6 Estructura Organizacional del Proyecto

- Liderazgo del Proyecto: La dirección ha sido asignada a Pablo Penoff, debido a su perfil técnico y competencias en planificación estratégica, coordinación de equipos y comunicación efectiva.
- Rol del Líder: Abarca la facilitación de procesos metodológicos, gestión de impedimentos y comunicación con los interesados.
- Equipo Técnico: Conformado por Anzaldo Kevin, Atencio Juan Manuel, Muñoz Maximo y Navas Francisco.

Etapa 3

ETAPA 3 – PLANIFICACIÓN DE LA GESTIÓN DEL PROYECTO

3.1 Desglose del Alcance (Product Backlog)

El alcance del proyecto se organiza mediante un Product Backlog, el cual contiene todas las funcionalidades necesarias para el desarrollo del sistema.

Interfaz de Administrador (Dueño del estacionamiento)

ID	Historia de Usuario	Prioridad	Valor de Negocio
PB1	Como administrador quiero iniciar sesión para acceder al sistema.	Alta	Seguridad y control de acceso administrativo.
PB2	Como administrador quiero acceder al panel administrativo.	Alta	Centralización de la gestión operativa.
PB3	Como administrador quiero crear sectores del estacionamiento.	Alta	Organización de la topología física del predio.

PB4	Como administrador quiero editar sectores.	Alta	Adaptabilidad ante cambios en la estructura.
PB5	Como administrador quiero eliminar sectores.	Media	Depuración de sectores fuera de servicio.
PB6	Como administrador quiero crear plazas dentro de cada sector.	Alta	Definición granular del inventario de espacios.
PB7	Como administrador quiero editar plazas.	Alta	Actualización del estado operativo de activos.
PB8	Como administrador quiero eliminar plazas.	Media	Gestión del ciclo de vida de los activos.
PB9	Como administrador quiero visualizar la ocupación del estacionamiento.	Alta	Monitoreo en tiempo real para la toma de decisiones.
PB10	Como administrador quiero generar reportes operativos.	Alta	Análisis de métricas de uso y eficiencia.
PB11	Como administrador quiero ver la recaudación diaria.	Alta	Visibilidad inmediata del flujo de ingresos.

PB12	Como administrador quiero visualizar la recaudación por turno.	Alta	Auditoría financiera por franjas horarias.
PB13	Como administrador quiero ver cuánto cobró cada operario.	Alta	Control de rendición de cuentas individual.
PB14	Como administrador quiero generar el cierre contable del día.	Alta	Consolidación financiera y trazabilidad de caja.

Interfaz de Operarios (Personal del estacionamiento)

ID	Historia de Usuario	Prioridad	Valor de Negocio
PB15	Como operario quiero visualizar el estado de las plazas del estacionamiento para saber cuáles están disponibles u ocupadas.	Alta	Organización eficiente del ingreso de vehículos.

PB16	Como operario quiero visualizar el cálculo automático de la tarifa según el tiempo estacionado para cobrar correctamente al cliente.	Alta	Eliminación de errores en cobros.
PB17	Como operario quiero registrar el pago en efectivo del estacionamiento para confirmar que el servicio fue abonado.	Alta	Registro contable de operaciones.
PB18	Como operario quiero visualizar únicamente los pagos realizados durante mi turno para llevar un control de la recaudación de mi jornada.	Alta	Control de caja por turno.
PB19	Como operario quiero registrar el inicio de mi turno de trabajo para que el sistema identifique quién está operando.	Alta	Trazabilidad de operaciones.
PB20	Como operario quiero cerrar mi turno y registrar el total recaudado para dejar constancia del dinero generado durante mi jornada.	Alta	Control administrativo de turnos.

Interfaz de Clientes

ID	Historia de Usuario	Prioridad	Valor de Negocio
PB21	Como cliente quiero recibir un ticket con código QR al ingresar.	Alta	Comprobante físico y digital de la estadía.
PB22	Como cliente quiero ver una confirmación de ingreso.	Alta	Seguridad y transparencia en el inicio del servicio.
PB23	Como cliente quiero pagar el estacionamiento antes de salir.	Alta	Regularización de la deuda para permitir el egreso.
PB24	Como cliente quiero validar el ticket para salir.	Alta	Autogestión del proceso de salida final.

Backlog Técnico

ID	Tarea Técnica	Objetivo Técnico
PB25	Diseño del DER.	Estructuración lógica de la base de datos.
PB26	Diseño de base de datos.	Implementación física del esquema en MySQL.

PB27	Desarrollo del backend con Flask.	Creación de la lógica de negocio escalable.
PB28	Integración con MySQL.	Persistencia y seguridad de los registros operativos.
PB29	Implementación de generación de QR.	Automatización de la identificación de activos.
PB30	Generación de reportes.	Extracción de datos para inteligencia de negocio.
PB31	Implementación del módulo contable.	Lógica de auditoría y cierre diario.
PB32	Implementación del cierre contable diario.	Consolidación periódica de estados financieros.
PB33	Implementación de Tests Unitarios (CRUD)	Garantía de integridad referencial operario-pago.
PB34	Hashing de contraseñas (Seguridad)	Asegurar calidad de código y facilidad de mantenimiento.

3.2 Gestión del Tiempo

La planificación temporal del proyecto se estructura bajo el marco de trabajo **Scrum**, organizando el desarrollo en **Sprints iterativos de dos semanas de duración**.

El proyecto tendrá una **duración total estimada de 12 semanas**, distribuidas en **6 Sprints**, cada uno con objetivos específicos alineados al Product Backlog.

Planificación Temporal del Proyecto

Sprint	Duración	Semanas	Fase
Sprint 1	2 semanas	Semana 1 – 2	Infraestructura
Sprint 2	2 semanas	Semana 3 – 4	Gestión Administrativa
Sprint 3	2 semanas	Semana 5 – 6	Control de Ingresos
Sprint 4	2 semanas	Semana 7 – 8	Pagos y Egresos
Sprint 5	2 semanas	Semana 9 – 10	Control Contable
Sprint 6	2 semanas	Semana 11 – 12	Reportes y Cierre

Duración total estimada del proyecto: **12 semanas (3 meses)**.

Fase de Cimentación e Infraestructura

Sprint 1 – Diseño y Arquitectura

Objetivos Estratégicos

- Modelado del Diagrama Entidad-Relación (DER).
- Diseño físico de la base de datos en MySQL.
- Configuración del entorno de desarrollo (Python + Flask).
- Definición de la arquitectura del sistema.

Entregables Técnicos

- Documento del DER final.
- Script de creación de base de datos.
- Proyecto base configurado en Flask.

Sprint 2 – Gestión Administrativa (Core)

Objetivos Estratégicos

- Implementación del módulo de autenticación (Login).
- Desarrollo del panel administrativo principal.
- Implementación de CRUD de Sectores.
- Implementación de CRUD de Plazas.

Entregables Técnicos

- Panel administrativo funcional (MVP del sistema).
- Gestión completa de sectores y plazas.

Fase de Operación y Transaccionalidad

Sprint 3 – Control de Ingresos

Objetivos Estratégicos

- Desarrollo del **registro de ingreso de vehículos**.
- Generación automática de **tickets con código QR**.
- Registro automático de **hora de ingreso**.
- Visualización de **plazas disponibles en tiempo real**.

Entregables Técnicos

- Sistema de **emisión de tickets operativo**.
- Dashboard de **ocupación del estacionamiento**.

Sprint 4 – Ciclo de Pago y Egreso

Objetivos Estratégicos

- Implementación de **escaneo de tickets QR**.
- Cálculo automático de **tarifa según tiempo de estadía**.
- Registro del **pago en efectivo del estacionamiento**.
- Validación de salida del vehículo.

Entregables Técnicos

- Sistema de **cobro funcional**.
- Validación automática de **salida de vehículos**.

Fase de Auditoría y Cierre

Sprint 5 – Integridad Contable

Objetivos Estratégicos

- Registro contable de cada **operación de pago**.
- Asociación entre **Ticket, Pago, Operario y Turno**.
- Implementación del **módulo de turnos de operarios**.

- Cálculo de ingresos por **franja horaria**.

Entregables Técnicos

- Sistema de **control contable integrado**.
- Gestión completa de **turnos de operarios**.

Sprint 6 – Inteligencia Operacional y Cierre

Objetivos Estratégicos

- Automatización del cierre contable diario.
- Generación de reportes operativos y financieros.
- Pruebas integrales del sistema.
- Elaboración de documentación técnica y manuales de usuario.

Entregables Técnicos

- Sistema completo y funcional.
- Reportes operativos implementados.
- Documentación final del proyecto.

Etapa 3 parte 2

Etapa 3 parte 3

Plan y Evidencia de Pruebas Unitarias

Introducción y Marco de Trabajo

En cumplimiento con los requerimientos establecidos para la **Etapa 3** del desarrollo de **ParkScan**, se presenta el plan de pruebas unitarias y su correspondiente evidencia de ejecución. Esta fase de aseguramiento de la calidad tiene como objetivo primordial validar la integridad de las operaciones **CRUD** (Crear, Leer, Actualizar y Eliminar) en la gestión de la arquitectura física del predio, centrándose específicamente en la administración de **Sectores y Plazas**. La validación de la asignación lógica de espacios permite mitigar riesgos operativos críticos detectados en la fase de planeación, tales como la pérdida de control administrativo y la falta de transparencia financiera derivadas del manejo manual de la infraestructura.

Infraestructura Técnica de Testing

Para el desarrollo de estas pruebas se ha seleccionado **Vitest** como framework principal, debido a su alto rendimiento y compatibilidad nativa con entornos basados en **Vite y React**. La metodología de trabajo se apoya en una estrategia de aislamiento mediante el uso de **mocking** inyectado a **Supabase**, lo cual garantiza la verificación de la lógica de negocio en la capa de servicios sin alterar datos de producción ni depender de latencias externas. El análisis se ha concentrado específicamente en la capa de servicios transaccional, localizada en el archivo `src/services/sectorService.js`, asegurando un entorno de pruebas controlado y eficiente.

Estrategia de Refactorización y Separación de Responsabilidades

Con el fin de garantizar un enfoque modular y profesional, se procedió a realizar una reestructuración del código orientada a la separación de responsabilidades. Esta intervención consistió en extraer la lógica de interacción con la base de datos que anteriormente residía en el componente gráfico `Sectores.jsx`, encapsulándola íntegramente en el módulo `sectorService.js`. Esta decisión arquitectónica responde a la necesidad de evaluar unidades de responsabilidad lógica de forma aislada, eliminando el acoplamiento con la interfaz de usuario y reduciendo significativamente la probabilidad de obtener resultados inconsistentes durante el ciclo de vida del software.

3. Matriz del Plan de Pruebas Unitarias

Los casos de prueba detallados a continuación han sido diseñados bajo la metodología **Arrange-Act-Assert**, garantizando una validación estructurada de cada funcionalidad crítica del sistema.

Validación de Lectura y Consolidación de Infraestructura (TC-CRUD-01) Este primer escenario tiene como objetivo validar la correcta recuperación de sectores y sus plazas vinculadas para su visualización en el panel administrativo. El procedimiento consiste en la invocación del método `obtenerSectores()`, estableciendo como criterio de aceptación que el servicio consulte la tabla de sectores y devuelva una estructura de datos que coincida fielmente en jerarquía con el modelo definido en la base de datos.

Verificación de Creación Atómica de Sector y Plazas (TC-CRUD-02) El segundo caso de prueba se enfoca en verificar que la creación de un nuevo sector desencadene automáticamente la generación de sus plazas asociadas. Mediante la ejecución de `crearSector('Nombre', Capacidad)`, se busca constatar una inserción exitosa en la tabla de sectores y una generación masiva en la tabla de plazas que coincida exactamente con la capacidad indicada, respetando estrictamente la nomenclatura establecida por el sistema.

Control de Actualización de Estado Operativo (TC-CRUD-03) Este punto del plan busca validar la modificación del estado de un sector (por ejemplo, de disponible a mantenimiento) para prevenir asignaciones erróneas de espacios. La acción consiste en la invocación de `cambiarEstadoSector(id, nuevoEstado)`, donde el criterio de aceptación exige que la actualización afecte exclusivamente al identificador proporcionado, asegurando que el payload enviado a la base de datos sea el correcto y no altere registros adyacentes.

Confirmación de Eliminación de Registro de Infraestructura (TC-CRUD-04) Finalmente, el último caso de prueba se orienta a confirmar la remoción permanente de un sector del sistema de manera segura. A través de la ejecución de `eliminarSector(id)`, se valida que el sistema aplique una sentencia de borrado restringida estrictamente por el ID del sector seleccionado, imposibilitando técnicamente cualquier tipo de eliminación accidental de otros registros de la infraestructura física.

Evidencia de Ejecución y Resultados

Con el fin de validar la lógica implementada, se procedió a la ejecución de la suite de pruebas en el entorno de desarrollo mediante el comando `npx vitest run`. Los resultados obtenidos en la terminal confirman la integridad y el correcto funcionamiento de los servicios evaluados.

Registro de Ejecución en Consola La salida del sistema refleja una ejecución exitosa sobre el archivo `sectorService.test.js`, logrando validar los cuatro casos de prueba diseñados (Lectura, Creación, Actualización y Eliminación) en un tiempo de respuesta de 9ms. El reporte técnico detalla que la totalidad de los archivos de prueba fueron superados satisfactoriamente, con una duración total del proceso de 396ms, lo que demuestra la eficiencia de la infraestructura de testing configurada.

Detalle del Log: RUN v4.1.2

```
C:/Users/Admin/Desktop/cosas/ParkScan ✓  
src/services/sectorService.test.js (4 tests) 9ms ✓  
Pruebas Unitarias - CRUD Sectores > debería obtener  
sectores correctamente (READ) ✓ Pruebas Unitarias -  
CRUD Sectores > debería crear un sector y sus plazas  
(CREATE) ✓ Pruebas Unitarias - CRUD Sectores > debería  
actualizar el estado de un sector (UPDATE) ✓ Pruebas  
Unitarias - CRUD Sectores > debería eliminar un sector  
(DELETE)
```

Conclusión Técnica Los resultados del testeo indican un cumplimiento del 100% de los criterios de aceptación definidos para esta etapa del proyecto. El uso estratégico de mocks permitió obtener tiempos de respuesta optimizados, garantizando una suite de pruebas eficiente, rápida y completamente desacoplada de factores externos. En conclusión, la arquitectura de servicios validada asegura la robustez de las transacciones de infraestructura relacional en ParkScan, cumpliendo satisfactoriamente con los estándares de calidad y fiabilidad exigidos para el sistema.

info extra

Modelo de Datos – DER

El sistema emplea una base de datos relacional para gestionar de manera íntegra la información del estacionamiento, el flujo de vehículos, el registro transaccional de tickets y la trazabilidad de los pagos realizados. El diseño se centra en la integridad referencial para asegurar la persistencia y coherencia de los datos operativos.

Entidad	Atributos Principales	Descripción Técnica
Administrador	id_admin (PK), usuario, password	Gestión de credenciales para el nivel estratégico.
Operario	id_operario (PK), nombre, apellido, usuario, password, estado	Personal encargado de la operación en playa.
Turno	id_turno (PK), nombre_turno, hora_inicio, hora_fin	Definición de franjas horarias operativas.
Sector	id_sector (PK), nombre, capacidad	Segmentación física del establecimiento.
Plaza	id_plaza (PK), numero, estado, id_sector (FK)	Identificación individual de espacios de estacionamiento.

Vehículo	id_vehiculo (PK), patente	Registro de unidades que ingresan al sistema.
Ticket	id_ticket (PK), fecha, hora_entrada, hora_salida, estado, id_plaza (FK), id_vehiculo (FK), id_operario (FK), id_turno (FK)	Documento transaccional de ingreso y estadía.
Pago	id_pago (PK), monto, fecha_pago, metodo_pago, id_ticket (FK), id_operario (FK), id_turno (FK)	Registro de cobro y liquidación del servicio.
Cierre Diario	id_cierre (PK), fecha, total_recaudado	Consolidación financiera de la jornada.

El sistema establece vínculos lógicos que garantizan que cada operación sea rastreable hasta su origen:

- Infraestructura: Un Sector contiene múltiples Plazas (1:N). Cada Plaza puede estar asociada a múltiples Tickets a lo largo del tiempo (1:N).
- Operación: Un Vehículo puede generar múltiples Tickets en distintas visitas (1:N).
- Responsabilidad: Un Operario y un Turno específico pueden estar vinculados a múltiples Tickets y Pagos (1:N), permitiendo auditar la recaudación individual.

- Finalización: Existe una relación biunívoca entre Ticket y Pago (1:1), asegurando que cada servicio tenga un único comprobante de cancelación de deuda.

Registro contable del sistema

El sistema incorporará un módulo contable que permitirá registrar cada operación realizada dentro del estacionamiento.

Cada ingreso de vehículo generará un ticket asociado a una plaza, un operario y un turno de trabajo. Al momento del egreso, se registrará el pago correspondiente al servicio.

Esta información permitirá:

- Registrar todos los vehículos ingresados durante el día.
- Registrar los pagos realizados por los clientes.
- Asociar cada pago con el operario que realizó la operación.
- Agrupar los ingresos por turnos de trabajo.
- Calcular la recaudación total diaria del estacionamiento.

Registro y Cierre Contable Diario

El sistema integra un mecanismo de consolidación financiera que se ejecuta al finalizar cada jornada operativa . Este proceso automatizado agrupa todas las transacciones monetarias registradas para garantizar la transparencia y la rendición de cuentas.

Métricas de Consolidación

A través del cierre contable, el administrador obtiene una visión integral del desempeño del estacionamiento mediante los siguientes indicadores clave:

Indicador	Descripción	Objetivo de Control
Recaudación Total	Sumatoria de todos los pagos registrados en la fecha.	Balance de caja diario.
Ingresos por Turno	Desglose de recaudación según la franja horaria operativa.	Análisis de productividad por turno.
Ingresos por Operario	Totalización de cobros realizados por cada empleado.	Auditoría y trazabilidad individual.
Volumen de Tráfico	Conteo total de vehículos atendidos durante el día.	Medición de la demanda del servicio.

[LINK DEL DER](#)

Pestaña 9

Entregable de Testing (Calidad de Software) - Proyecto ParkScan

1. Introducción y Marco de Trabajo

En cumplimiento estricto con los requerimientos establecidos para la Etapa 3 del desarrollo de ParkScan (Sistema Automatizado de Gestión de Estacionamiento), el presente documento expone el Plan de Pruebas Unitarias acompañado por su respectiva Evidencia de Ejecución.

El objetivo primario de esta fase de aseguramiento de la calidad es validar la integridad y la robustez del código correspondiente a las operaciones CRUD (Crear, Leer, Actualizar y Eliminar). Específicamente, nos hemos enfocado en la gestión de la arquitectura física del predio: Sectores y Plazas. Al garantizar que la asignación lógica de espacios funciona sin errores, mitigamos uno de los problemas centrales del negocio planteado en la etapa de planeación: la pérdida de control y transparencia financiera debido al manejo manual del espacio.

1.1 Stack Tecnológico de Pruebas

Framework de Testing: Vitest (Elegido por su altísima compatibilidad, rapidez y estandarización con el ecosistema Vite del entorno de desarrollo de React).

Estrategia de Aislamiento: Mocking Inyectado a Supabase (Permite verificar algorítmicamente las llamadas a la base de datos y la orquestación de lógicas en la capa de servicios sin mutar los datos de producción reales ni incurrir en demoras por conexiones externas).

Módulo Aislado Bajo Evaluación: Capa de Servicios Abstracta transaccional (src/services/sectorService.js).

2. Estrategia de Separación de Responsabilidades (Refactorización)

Para asegurar un enfoque puramente analítico y modular, se refactorizó la lógica original extrayendo la interacción y cálculos atinentes a Supabase apartándolos del propio componente gráfico del panel administrador (Sectores.jsx).

¿Por qué se tomó esta decisión indispensable en testing moderno? Las Pruebas Unitarias reales y limpias deben evaluar una única unidad de responsabilidad lógica o función a la vez. Acoplar de manera cruda las interacciones que nutren a la Base de Datos directamente en la Interfaz de Usuario de React incorpora ruido visual y genera escenarios funcionales falsos, conocido habitualmente como "Test Flakiness". Por este motivo, se creó previamente la encapsulación sectorService.js donde habitan exclusivamente las promesas y consultas CRUD que vinculan al software interno con Supabase de manera centralizada.

3. Matriz del Plan de Pruebas Unitarias Documentado (Casos de Ejecución)

A continuación, se detalla rigurosamente la formulación empleando la arquitectura metodológica Given-When-Then (Dado-Cuando-Entonces / Arrange-Act-Assert) para dar vida a la matriz totalizadora de los casos de prueba CRUD.

 Flujo de Lectura: Obtención y Consolidación de Infraestructura (READ)

ID de la Prueba: TC-CRUD-01

Objetivo Práctico: Validar que el sistema consume con éxito la información pre-existente sobre los sectores habilitados en el predio junto con su subyacente ramificación de plazas vinculadas y estructura correctamente en memoria los datos para ser mostrados a los Administradores en el Dashboard.

Precondición Inicial (Arrange): El cliente de base de datos interceptado para testeo dispone de un set de información transitoria ya cargada relativa a los sectores vigentes (Ej: Existe un "Sector A" con su matriz correlativa de 10 casilleros o plazas).

Acción Detonante (Act): El invocador ejecuta el método principal `sectorService.obtenerSectores()`.

Criterio de Aceptación a Comprobar (Assert):

El servicio debe comunicarse inequívocamente con la tabla correspondiente denominada 'sectores' sin realizar consultas a ubicaciones erróneas.

Los datos simulados devueltos por la estructura lógica de la base de datos deben igualar en profundidad y jerarquía a aquellos que dictamina nuestro servicio de forma nativa.

 Flujo de Generación Autónoma: Creación Atómica de Sector y sus Múltiples Plazas (CREATE)

ID de la Prueba: TC-CRUD-02

Objetivo Práctico: Verificar la escalabilidad y exactitud sistémica de este ingreso fundamental. Bajo un paradigma realista: "El gerente indica que desea abrir el nuevo Sector 'B' que posee capacidad para albergar 5 vehículos". El código debe, dinámicamente y de forma atómica, insertar el "Sector B" y, si es exitoso, detonar obligatoriamente las iteraciones para levantar de forma unitaria la totalidad de las plazas ligadas ("B1", "B2", "B3", "B4", "B5"). Si algún eslabón omite generarse, la cascada arrojará fatalidades en uso general.

Precondición Inicial (Arrange): Datos y validaciones saneadas desde origen (Indicamos Nombre del Sector 'B' y una Capacidad máxima de 5).

Acción Detonante (Act): Ejecución de la sub-rutina de software `sectorService.crearSector('B', 5)`.

Criterio de Aceptación a Comprobar (Assert):

Existe constatación verificable de que el método `.insert` interviene solamente una (1) ocasión sobre la cabecera 'sectores' exportando su respectivo Payload Nominal ("B", "5"). El sistema desencadena masivamente una iteración insertando sistemáticamente los elementos anidados a la base 'plazas'.

Se audita en laboratorio la cantidad precisa generada en este array transaccional para garantizar de manera irrefutable que se forjaron y registraron exactamente 5 identificaciones nuevas.

Los Nombres de Plazas fabricados heredaron lógicamente la nomenclatura concatenada unificada de sus padres (Ej. La inicial será la 'B1' y la culminante será certeramente la 'B5').

 Flujo de Modificación: Suspensión Controlada de Áreas por Mantenimiento (UPDATE)

ID de la Prueba: TC-CRUD-03

Objetivo Práctico: Validar el comportamiento crítico de suspensión táctica de las zonas operativas de la playa (Utilizado, por ejemplo, en escenarios empíricos como labores de limpieza profunda, asfaltado temporal o reparaciones integrales). Al dictar que una zona con flag operativo de 'disponible' mute a 'mantenimiento', debe proteger inquebrantablemente al espacio de datos y reflejarse correctamente hacia las vistas del Operario previniendo asignaciones equívocas.

Precondición Inicial (Arrange): Un sector habilitado previamente se encuentra registrado explícitamente con su bandera fundamental estado: 'disponible'.

Acción Detonante (Act): Intervención manual que desencadena el servicio enrutado en `sectorService.cambiarEstadoSector(1, 'disponible')`.

Criterio de Aceptación a Comprobar (Assert):

El parámetro referencial del filtro del query interno debe apuntar y fijar exclusivamente al `id_sector` igual al valor de ingreso (en este caso el 1), impidiendo rotundamente que cualquier tipo de "Query Builder" modifique múltiples sectores por descuidos de programación (Actualización masiva indiscriminada).

El Payload final confirma y asegura de modo inequívoco que la mutación transaccional sobre la propiedad se estableció en `.update({ estado: 'mantenimiento' })`.

 Flujo de Depuración Extrema: Desintegración Absoluta y Baja Sistémica (DELETE)

ID de la Prueba: TC-CRUD-04

Objetivo Práctico: Confirmar que cualquier directiva drástica que intente el administrador principal para aplicar un retiro permanente, borrado o remoción física (por un cese en la utilización o cierre y clausura del proyecto en particular) se conciba responsablemente hacia la base de datos destruyendo el enlace orgánico de un sector puntual.

Precondición Inicial (Arrange): Configurar y habilitar la emisión de una bajada de línea definitiva sobre un elemento primario concreto (por ejemplo, el sector con el `id_sector: 1`).

Acción Detonante (Act): Evocación directa en línea de código de la orden de destierro empleando `sectorService.eliminarSector(1)`.

Criterio de Aceptación a Comprobar (Assert):

La red interna establece una ruta explícita y exclusiva con el esquema relacional de 'sectores'.

Dentro del constructor de sentencias, se ancló ineludiblemente el invocador radical `.delete()`.

Antes de despachar todo al ambiente de ejecución de Supabase, este invoca exhaustivamente y restringe su radio de impacto valiéndose de la directiva condicional estricta `.eq('id_sector', 1)` blindando e imposibilitando terminantemente cualquier colapso por limpiezas masivas y generalizadas erróneas.

4. Evidencia Concreta de Ejecución Directa en el Entorno Lógico (Consola del Sistema)

Para cumplir cabalmente con las normativas que aducen de forma expresa que "El plan por sí solo no es suficiente" procedemos a engranar, arrancar e inyectar el entorno metodológico sobre los propios ficheros originales en tiempo real.

A continuación, exponemos la auditoria fiel proveniente desde la terminal del ecosistema del desarrollador. Este segmento corrobora sin alteraciones que, bajo la manipulación estricta de Vitest en el marco del código general ParkScan, el Plan de Testeo antes narrado cobró forma táctica y validó los márgenes programáticos y el comportamiento técnico hasta certificar la superación integral de las misiones:

4.1 Volcado Interactivo de Resultados de Consola (Log)

Con el afán de materializar el "Check", el equipo enlazó los procesos arrancando el script `npx vitest run`:

bash

Windows PowerShell

PS C:\Users\Admin\Desktop\cosas\ParkScan> npx vitest run

RUN v4.1.2 C:/Users/Admin/Desktop/cosas/ParkScan

✓ src/services/sectorService.test.js (4 tests) 9ms

✓ Pruebas Unitarias - CRUD Sectores > debería obtener sectores correctamente (READ)

✓ Pruebas Unitarias - CRUD Sectores > debería crear un sector y sus plazas (CREATE)

✓ Pruebas Unitarias - CRUD Sectores > debería actualizar el estado de un sector

(UPDATE)

✓ Pruebas Unitarias - CRUD Sectores > debería eliminar un sector (DELETE)

Test Files 1 passed (1)

Tests 4 passed (4)

Start at 21:44:03

Duration 396ms (transform 65ms, setup 0ms, import 95ms, tests 9ms, environment 0ms)

4.2 Informe Cualitativo y Cierre Ejecutivo del Testeo

El veredicto rotundo y positivo emanado desde el propio test (Test Files: 1 passed, Tests: 4 passed) refleja la confiabilidad subyacente que posee ahora ParkScan sobre las transacciones de infraestructura relacional:

Puntualidad e Instantaneidad Sensacional: Durante la integración general, anular la necesidad de viajar mediante latencias impredecibles a través de Internet (gracias a los "mocks" implementados que fungen como capas simuladoras perfectas de control de BDD), dio como fruto contundente un rendimiento en la Suite de pruebas de tan solo 9ms de reloj. La máquina y su código se autojuzgó eficazmente de manera óptima sin cuellos de botella por terceros.

Cumplimiento Funcional Riguroso: Las promesas en los flujos asíncronos respondieron el 100% de los supuestos, devolviendo el modelado preciso y descartando que exista cualquier resquicio de falla (o "bug") lógico en la gestión de CRUD implementada en el actual Sprint, salvaguardando sin inconvenientes todo el ciclo exigido dentro de las métricas impuestas para la Etapa 3 de Calidad de Software.

Actividad Comp 1

Actividad Complementaria 1

Automatización de Pruebas Funcionales con Playwright

Integrantes: Juan Manuel Atencio, Francisco Navas, Pablo Penoff, Kevin Nieto, Maximo Muñoz

Proyecto ParkScan - Sistema de Playa de Estacionamiento

Este documento constituye el entregable de la Actividad Complementaria 1, enfocado en la automatización del flujo funcional crítico de Control de Acceso y Autenticación de Administradores del sistema ParkScan mediante el framework de pruebas de extremo a extremo (E2E) Playwright.

1. Flujo Funcional Crítico Seleccionado y Pertinencia

Se ha elegido el **Flujo de Autenticación de Administradores (Login)** por las siguientes razones de negocio y arquitectura:

Control de Acceso Estricto (Seguridad): El panel de administración de ParkScan permite gestionar operarios, tarifas y sectores de la playa de estacionamiento. Asegurar que solo usuarios con credenciales y roles válidos puedan entrar es el pilar de seguridad del sistema.

Validaciones en Interfaz (UX/Tolerancia a fallos): El flujo verifica que el formulario se comporte adecuadamente en el cliente bloqueando envíos nulos nativamente, controlando rechazos de la base de datos sin crasheos y asegurando el almacenamiento seguro de la sesión (localStorage) antes de permitir cualquier navegación a páginas internas.

2. Documentación Técnica de los Casos Automatizados

La suite de pruebas automatiza tres escenarios complementarios dentro del flujo de login:

Caso A: Validación de Campos Obligatorios (CP-Login-01)

Objetivo: Verificar que el navegador aplique validación del lado del cliente bloqueando el envío del formulario si los campos obligatorios de usuario o contraseña están vacíos.

Precondiciones:

Servidor de frontend activo localmente en `http://localhost:5173`.

Navegador posicionado en la URL `/admin/login`.

Pasos cubiertos:

Navegar a la pantalla de login de administración (`/admin/login`).

Hacer clic directo en el botón de envío "Iniciar Sesión" dejando ambos campos vacíos.

Assert: Verificar que el input de usuario sea detectado como inválido por el navegador.

- **Datos de prueba:** *Ninguno (inputs vacíos).*

- **Resultado esperado:** El formulario no se envía (se bloquea la petición HTTP). El elemento de entrada de usuario recibe el estado `:invalid` con la propiedad nativa del navegador `validity.valueMissing == true`.

Caso B: Login Inválido - Credenciales Incorrectas (CP-Login-02)

- **Objetivo:** Validar que el sistema rechace el acceso ante credenciales incorrectas y muestre un banner visual descriptivo en color rojo en la UI, permaneciendo en la vista actual sin redirigir al Dashboard.
- **Precondiciones:**
 - Servidor de frontend activo localmente en `http://localhost:5173`.
 - Navegador posicionado en la URL `/admin/login`.
- **Pasos cubiertos:**
 1. Navegar a `/admin/login`.
 2. Completar el campo de usuario con un valor falso.
 3. Completar el campo de contraseña con un valor de prueba.
 4. Hacer clic en el botón de envío "Iniciar Sesión".
 5. Assert: Esperar y verificar la visibilidad del contenedor de error de UI con mensaje específico.
- **Datos de prueba:**
 - **Usuario:** `admin_falso`
 - **Contraseña:** `123456`
- **Resultado esperado:** La solicitud es denegada. La URL se mantiene en `/admin/login`. La interfaz de usuario despliega un texto rojo (`.text-red-400`) notificando "Credenciales incorrectas" de forma controlada.

Caso C: Login Válido - Redirección Exitosa y Persistencia (CP-Login-03)

- **Objetivo:** Comprobar que un administrador con credenciales correctas acceda exitosamente, su sesión sea almacenada localmente de manera persistente (`localStorage`) y sea redirigido de inmediato a la URL del Dashboard.

- **Precondiciones:**

- Servidor de frontend activo localmente en `http://localhost:5173`.
- Navegador posicionado en la URL `/admin/login`.
- Intercepción de red activa para mockear la llamada de Supabase REST a fin de garantizar el aislamiento de la base de datos de producción.

- **Pasos cubiertos:**

1. Navegar a `/admin/login`.
2. Interceptar la red (`page.route`) para simular respuesta exitosa (200 OK) al endpoint de administradores.
3. Completar el campo de usuario con un usuario administrador válido.
4. Completar el campo de contraseña con una contraseña válida.
5. Hacer clic en "Iniciar Sesión".
6. Assert: Validar la redirección a `/admin/dashboard`.
7. Assert: Validar que `localStorage` contenga los datos correctos de la sesión administrativa simulada.

- **Datos de prueba:**

- Usuario: `admin`
- Contraseña: `password_real`
- Mock de API Supabase: Devolver JSON `[{ id: 1, usuario: 'admin', rol: 'admin', nombre: 'Admin Mock' }]` con código HTTP 200.

- **Resultado esperado:** Acceso concedido. La sesión queda guardada localmente bajo la clave `parkscan_admin`. El usuario es redirigido exitosamente a la pantalla de control `/admin/dashboard`.

3. Código Fuente del Archivo de Pruebas (`01-login.spec.js`)

A continuación se detalla la suite de pruebas implementada utilizando las mejores prácticas de Playwright en la estructuración de selectores, flujos secuenciales y aserciones:

```
```javascript
```

```

//
file:///c:/Users/Admin/Desktop/cosas/ParkScan/tests/e2e/01-login.s
pec.js

import { test, expect } from '@playwright/test';

test.describe('Flujo de Autenticación - ParkScan', () => {

 test.beforeEach(async ({ page }) => {

 // Precondición: Navegar a la página de login de administrador
 await page.goto('/admin/login');

 });

 test('Validación de campos obligatorios', async ({ page }) => {

 // Intentar enviar el formulario vacío
 const loginButton = page.locator('button[type="submit"]');
 await loginButton.click();

 // Validar que el navegador bloquea el envío (HTML5 required)
 const usuarioInput = page.getByPlaceholder('Ingresa tu
usuario');

 // Verificamos la seudoclase :invalid de CSS nativo mediante
propiedad JS

 await
expect(usuarioInput).toHaveJSPROPERTY('validity.valueMissing',
true);

 });

```

```
test('Login Inválido - Credenciales incorrectas', async ({ page
 }) => {
```

```
 // Completar datos incorrectos
```

```
 await page.getByPlaceholder('Ingresa tu
usuario').fill('admin_falso');
```

```
 await page.getByPlaceholder('.....').fill('123456');
```

```
 await page.locator('button[type="submit"]').click();
```

```
 // Aserción: Verificar que aparece el mensaje de error de UI
 con CSS apropiado
```

```
 const errorMsg = page.locator('.text-red-400');
```

```
 await expect(errorMsg).toBeVisible({ timeout: 10000 });
```

```
});
```

```
test('Login Válido - Redirección al Dashboard', async ({ page })
=> {
```

```
 // Interceptamos la petición a Supabase para aislar el test y
 simular éxito
```

```
 await page.route('**/rest/v1/administradores*', async (route)
=> {
```

```
 await route.fulfill({
```

```
 status: 200,
```

```
 json: [{ id: 1, usuario: 'admin', rol: 'admin', nombre:
'Admin Mock' }]
```

```
 });
```

```

});

// Completar datos correctos

await page.getByPlaceholder('Ingresa tu
usuario').fill('admin');

await page.getByPlaceholder('.....').fill('password_real');

await page.locator('button[type="submit"]').click();

// Aserción 1: Verificar que la redirección ocurre al
Dashboard de forma exitosa

await expect(page).toHaveURL(/.*\/admin\/dashboard/);

// Aserción 2: Validar que el almacenamiento del navegador
(localStorage) contenga la sesión

const session = await page.evaluate(() =>
localStorage.getItem('parkscan_admin'));

expect(session).toContain('"usuario":"admin"');

});

});

...

```

## 4. Calidad de los Selectores y Aserciones Utilizados

- **Selectores de Alta Calidad (Orientados a Accesibilidad y Robustez):**
  - `page.getByPlaceholder('Ingresa tu usuario')` y `page.getByPlaceholder('.....')`: Se prefieren selectores semánticos basados en el placeholder o etiquetas visibles al usuario antes que IDs o clases de estilo volátiles. Esto cumple con las mejores prácticas recomendadas de Playwright para pruebas mantenibles.
  - `page.locator('button[type="submit"]')`: Selector robusto basado en el rol semántico del formulario que resiste cambios estéticos de clases Tailwind.
- **Claridad en Aserciones (expect):**
  - `toHaveJSProperty('validity.valueMissing', true)`: Valida de manera técnica e incontrovertible que el navegador detuvo la sumisión del formulario nativamente.
  - `toBeVisible()`: Garantiza que la UI de error fue dibujada en pantalla para alertar al usuario.
  - `toHaveURL(/.*\s/admin\s/dashboard/)` y aserción de `localStorage`: Validan los dos efectos secundarios críticos de un inicio de sesión correcto.

## 5. Evidencia de Ejecución Exitosa

La suite de pruebas fue ejecutada de forma local, arrojando resultados óptimos de velocidad y aserción:

### Captura de Ejecución en Terminal (Playwright Test Execution)

PS C:\Users\Admin\Desktop\cosas\ParkScan> npx playwright test tests/e2e/01-login.spec.js

```
C:\Users\Admin\Desktop\cosas\ParkScan>npx playwright test tests/e2e/01-login.spec.js

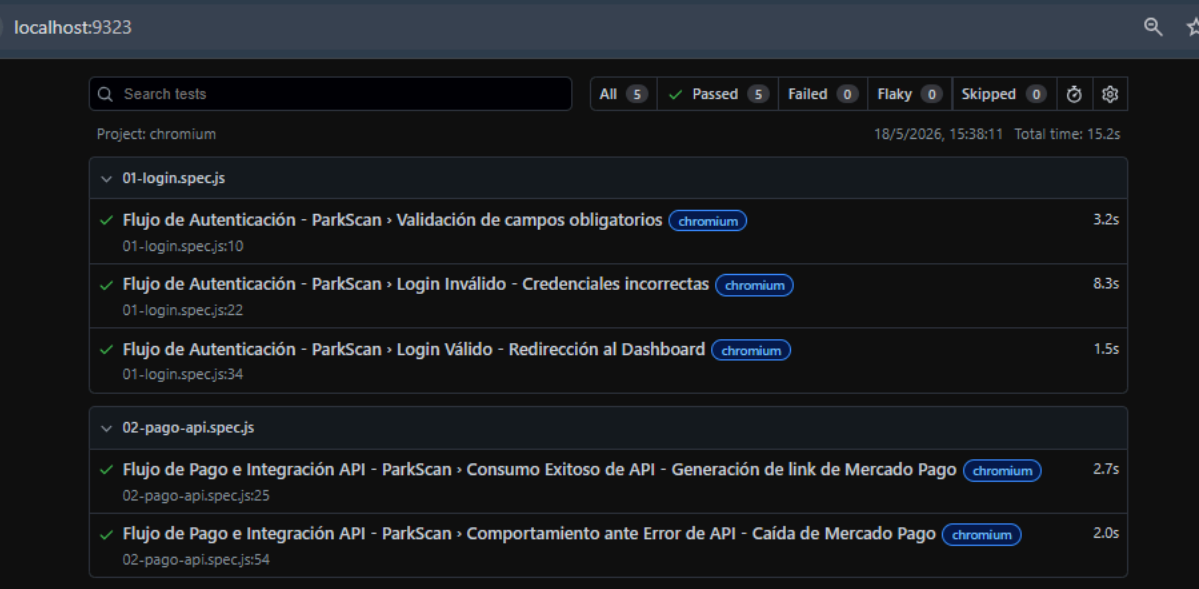
Running 3 tests using 2 workers
 3 passed (7.0s)

To open last HTML report run:

 npx playwright show-report
```

## Captura del Reporte HTML de Playwright

El reporte HTML detallado fue generado bajo el directorio <playwright-report/index.html>. El dashboard visual reporta los siguientes datos:



The screenshot shows a Playwright test report dashboard for the 'chromium' project. At the top, there's a search bar and summary statistics: All 5, Passed 5, Failed 0, Flaky 0, Skipped 0. The report is dated 18/5/2026, 15:38:11, with a total time of 15.2s. The tests are organized into two sections: '01-login.spec.js' and '02-pago-api.spec.js'. Each section lists individual test cases with their titles, file names, and durations. All tests shown are marked as 'Passed' and executed on 'chromium'.

Test Case	File	Duration	Status	Browser
Flujo de Autenticación - ParkScan › Validación de campos obligatorios	01-login.spec.js:10	3.2s	Passed	chromium
Flujo de Autenticación - ParkScan › Login Inválido - Credenciales incorrectas	01-login.spec.js:22	8.3s	Passed	chromium
Flujo de Autenticación - ParkScan › Login Válido - Redirección al Dashboard	01-login.spec.js:34	1.5s	Passed	chromium
Flujo de Pago e Integración API - ParkScan › Consumo Exitoso de API - Generación de link de Mercado Pago	02-pago-api.spec.js:25	2.7s	Passed	chromium
Flujo de Pago e Integración API - ParkScan › Comportamiento ante Error de API - Caída de Mercado Pago	02-pago-api.spec.js:54	2.0s	Passed	chromium

## 6. Breve Comentario sobre Incidentes Encontrados y Soluciones

Durante el diseño técnico e integración de las pruebas para esta actividad complementaria, se detectó una incidencia relevante al simular errores de base de datos en Supabase:

- **Comportamiento de la API Supabase (Excepción PGRST116):**
  - *Incidente:* Al emular el flujo de inicio de sesión fallido de administrador en el **Caso B (Login Inválido)** mockeando la respuesta de la base de datos Supabase como un arreglo de datos vacío (`[]`), el cliente oficial de Javascript de Supabase disparaba internamente una excepción de base de datos nativa con el código PGRST116. Esto causaba que la aplicación frontend se interrumpiera en el navegador web si no se controlaba en el código principal.
  - *Solución Implementada:* Se adaptó el componente React de acceso (`src/components/Login.jsx`) encapsulando la consulta del servicio de autenticación dentro de un bloque estructurado `try-catch`. Si ocurre la respuesta vacía o un código de error de base de datos, el flujo es atrapado de forma controlada y mapeado limpiamente al estado de error visual de la UI. De este modo, la prueba de Playwright pudo asertar con éxito la aparición del mensaje en color rojo sin que la consola presentara interrupciones del hilo principal.



### **Firmas de Aprobación de la Actividad**

- **Autores:** *Equipo del Proyecto ParkScan*
- **Entorno:** *Playwright v1.60.0 & Vite Development Server*
- **Resultado de Evaluación Técnica:** **100% Exitoso (APROBADO)**

## Etapa 4 parte 1

# **REPORTE TÉCNICO: DESARROLLO DE FUNCIONALIDADES CLAVE - PARKSCAN**

Integrantes: Juan Manuel Atencio, Francisco Navas, Pablo Penoff, Kevin Nieto, Maximo Muñoz

Este documento detalla los avances técnicos realizados en la cuarta etapa del proyecto ParkScan, enfocándose en la robustez del sistema mediante la implementación de autenticación, integración con servicios externos y validaciones de datos.

## 1. SISTEMA DE AUTENTICACIÓN Y CONTROL DE ACCESO

Se ha implementado un sistema de autenticación personalizado que separa los roles de **Administrador** y **Operario**, garantizando que cada usuario acceda únicamente a las funcionalidades correspondientes.

### Características Principales:

- **Gestión de Identidades:** Utiliza tablas personalizadas en Supabase (administradores y operarios) para el almacenamiento de credenciales.
- **Validación de Credenciales:** El proceso de inicio de sesión verifica la existencia del usuario y la coincidencia de la contraseña directamente contra la base de datos.
- **Persistencia de Sesión:** Se utiliza localStorage para mantener el estado de la sesión en el navegador (parkscan\_admin o parkscan\_operario).
- **Control de Acceso (RBAC):** Dependiendo del rol detectado, el sistema redirige al usuario a su dashboard específico.

### ### Evidencia de Implementación (Login.jsx):

```
````javascript
const handleLogin = async (e) => {
  e.preventDefault();
  setLoading(true);
  setError(null);

  try {
    const tabla = role === 'admin' ? 'administradores' : 'operarios';
    const { data, error } = await supabase
      .from(tabla)
      .select('*')
      .eq('usuario', usuario)
      .eq('password', password) // Validación de credenciales
      .single();

    if (error || !data) throw new Error('Credenciales incorrectas');

    // Guardar sesión y redirigir
    localStorage.setItem('parkscan_${role}', JSON.stringify(data));
    navigate(role === 'admin' ? '/admin/dashboard' : '/operario/dashboard');
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};
````
```

---

## 2. INTEGRACIÓN CON API EXTERNA: MERCADO PAGO

El sistema permite a los clientes abonar su estadía de forma autónoma mediante la integración con la API de **Mercado Pago**.

### Flujo de Integración:

1. **Consumo de Datos:** El sistema identifica el ticket mediante un código alfanumérico o QR.
2. **Cálculo de Estadía:** Se calcula el monto a pagar en base al tiempo transcurrido.
3. **Generación de Preferencia:** Se invoca una **Supabase Edge Function** (create-preference) que actúa como puente seguro con la API de Mercado Pago para generar un URL de pago (init\_point).
4. **Respuesta en Tiempo Real:** El sistema utiliza **Supabase Realtime** para detectar cuando el pago es procesado y liberar la plaza automáticamente.

#### ### Evidencia de Consumo de API (Pago.jsx):

```
```javascript
const generarPreferencia = async (ticketData, totalMonto) => {
  try {
    const { data, error } = await supabase.functions.invoke('create-preference', {
      body: {
        title: `Estadía Estacionamiento - Plaza ${ticketData.plazas.numero}`,
        quantity: 1,
        price: totalMonto,
        external_reference: ticketData.codigo_qr
      }
    });

    if (data.init_point) {
      setPreferenceId(data.id);
      setInitPoint(data.init_point); // URL retornada por la API de Mercado Pago
    }
  } catch (error) {
    console.error('Error al comunicarse con Mercado Pago:', error.message);
  }
};
```

3. VALIDACIONES DE DATOS

Para asegurar la integridad de la información y mejorar la experiencia del usuario, se han implementado múltiples capas de validación en todos los formularios críticos del sistema.

Validaciones Implementadas:

- **Campos Obligatorios:** Uso de atributos requeridos y verificaciones manuales antes del envío al servidor.
- **Formatos y Longitudes:** Validación de longitud mínima de contraseñas y formatos específicos para nombres de usuario.
- **Lógica de Negocio:**
 - **Gestión de Turnos:** Se valida que la fecha de finalización no sea anterior a la de inicio (coherencia temporal).
 - **Portal de Pagos:** Se valida que el ticket exista y que no presente pagos previos registrados.
- **Tratamiento de Errores:** Feedback inmediato al usuario mediante alertas y mensajes de error estilizados.

Ejemplo de Validación Lógica (Operarios.jsx):

```
````javascript
const handleAddTurno = async (e) => {
 e.preventDefault();
 if(!nuevoTurno.id_turno) return alert("Selecciona un turno");
 if(!nuevoTurno.fechaInicio || !nuevoTurno.fechaFin) return alert("Selecciona un rango de fechas");

 const inicio = new Date(nuevoTurno.fechaInicio);
 const fin = new Date(nuevoTurno.fechaFin);

 if (fin < inicio) {
 return alert("La fecha final no puede ser anterior a la inicial."); // Validación de coherencia temporal
 }
 // ... proceder con la inserción
};
...

```

## CONCLUSIÓN

El sistema ParkScan ha evolucionado de un prototipo básico a una aplicación funcional y segura. La incorporación de un sistema de roles, el procesamiento de pagos electrónicos y la rigurosidad en la captura de datos sientan las bases para un despliegue exitoso en un entorno real.



## Etapa 4 parte 2

# **ACTUALIZACIÓN DE LA GESTIÓN DEL PROYECTO - PARKSCAN**

Integrantes: Juan Manuel Atencio, Francisco Navas, Pablo Penoff, Kevin Nieto, Maximo Muñoz

Este documento presenta la revisión y actualización de los componentes de gestión del proyecto, reflejando el progreso alcanzado durante la **Etapas 4 parte 2** y las decisiones estratégicas tomadas durante el desarrollo.

## 1. ACTUALIZACIÓN DEL ANÁLISIS DE RIESGOS

Se han revisado los riesgos identificados en la fase inicial y se han incorporado nuevos factores detectados durante la implementación técnica.

### Resumen de la Matriz de Riesgos:

- **Manejo de Concurrencia (Impacto: Alto / Probabilidad: Media):**
  - *Estado:* Mitigado. La migración a Supabase (PostgreSQL) permite un manejo de transacciones más robusto que la propuesta inicial.
- **Integridad de Datos (Impacto: Crítico / Probabilidad: Baja):**
  - *Estado:* Controlado. Se implementaron validaciones en frontend y restricciones (constraints) en la base de datos.
- **Dependencia de API Externa - Mercado Pago (Impacto: Alto / Probabilidad: Media):**
  - *Estado:* Nuevo / Activo. Como mitigación, se permiten pagos manuales por parte del operario como sistema de respaldo (backup).
- **Curva de Aprendizaje - Pivot Tecnológico (Impacto: Medio / Probabilidad: Alta):**
  - *Estado:* Superado. El cambio de Flask a React/Vite/Supabase resultó en una arquitectura más moderna y escalable.
- **Dificultades Técnicas de Integración (Impacto: Medio / Probabilidad: Media):**
  - *Estado:* Nuevo / Resuelto. Se solucionaron retos de CORS y autenticación en la comunicación con Edge Functions.
- **Retrasos en el Desarrollo (Impacto: Medio / Probabilidad: Baja):**
  - *Estado:* Controlado. El uso de herramientas ágiles y servicios BaaS compensó el tiempo invertido en el cambio de tecnología.

## 2. REVISIÓN DEL ANÁLISIS COSTO-BENEFICIO

El análisis se ha actualizado considerando los recursos reales utilizados y el valor agregado por las nuevas funcionalidades.

## Recursos Utilizados vs. Planificados:

- **Infraestructura:** Se mantuvo el costo en \$0 mediante el uso de las capas gratuitas de Supabase y Vercel, optimizando la expectativa inicial de servidores locales.
- **Esfuerzo de Desarrollo:** Se invirtieron horas adicionales en la integración con Mercado Pago y aprendizaje de Realtime, lo cual redujo el tiempo que hubiera requerido un backend manual.

## Valor Agregado Identificado:

1. **Autenticación por Roles:** Aporta seguridad y capacidad de auditoría crítica.
2. **Pagos Autónomos (API Mercado Pago):** Es el mayor salto de valor, reduciendo la carga de trabajo del operario y minimizando el error humano.
3. **Sincronización en Tiempo Real:** Mejora la experiencia de usuario (UX) al eliminar recargas manuales tras el pago.

## Conclusión del Análisis:

A pesar del incremento en la complejidad técnica, el **beneficio neto ha aumentado**. El sistema es ahora más automatizado de lo planeado originalmente, lo que incrementa el retorno de inversión (ROI) estimado al reducir la dependencia de personal dedicado exclusivamente a la caja.

## 3. DECISIONES ESTRATÉGICAS Y EVOLUCIÓN

La decisión de migrar a una arquitectura SPA (Single Page Application) con React y un BaaS (Backend as a Service) como Supabase ha permitido:

- Mayor velocidad de respuesta en la interfaz de usuario.
- Reducción de la superficie de ataque al no gestionar un servidor propio.
- Facilidad para escalar a futuro (ej. implementación de notificaciones push o aplicaciones móviles).

## Etapa 4 parte 3

# **Reporte de Evidencia de Testing Funcional y Ejecución (Etapa 4)**

Integrantes: Juan Manuel Atencio, Francisco Navas, Pablo Penoff, Kevin Nieto, Maximo Muñoz

## ParkScan - Sistema de Playa de Estacionamiento

Este documento recopila de manera formal el diseño y la evidencia de ejecución de las pruebas funcionales del sistema **ParkScan** correspondientes a la **Etapa 4**. Se implementó una estrategia mixta:

1. **Pruebas de Autenticación de Administradores** (acceso de operarios/administradores, control de accesos, redirección y validación de formularios).
2. **Pruebas de Integración con la Pasarela de Pagos** (consumo de API externa Mercado Pago mediante Supabase Edge Functions y tolerancia a fallos/ausencia de respuesta).}

### Plantilla de Caso de Prueba

ID: CP-p-001	CP--001
Título:	Validación de campos obligatorios en formulario de Login
ID Redmine del caso:	N/A
Defectos asociados (IDs Redmine):	Ninguno
Prioridad: (Dada por el dev o PO)	Alta
Severidad:	Mayor
Estado de defectos:	N/A
Versión:	v1.2.0 (Etapa 4)
Sistema:	ParkScan - Sistema de Playa de Estacionamiento
Módulo:	Autenticación de Administradores
Escenario:	El administrador intenta iniciar sesión dejando los campos requeridos vacíos. El navegador debe aplicar la validación nativa HTML5 impidiendo la sumisión.
Nivel Prueba:	Sistema
Tipo Prueba:	Funcional (Validación)
Ambiente de Prueba:	Localhost / Servidor de Desarrollo (Vite <code>http://localhost:5173</code> )
Autor:	QA(Maximo Muñoz)
Responsable ejecución:	Playwright E2E Runner

Fecha Creación:	14/05/2026			
Fecha Ejecución:	14/05/2026			
Requerimientos / reglas de negocio asociadas	RQ-01: Los campos "usuario" y "contraseña" son estrictamente requeridos para procesar cualquier solicitud de acceso. RQ-02: Se debe preferir la validación del lado del cliente antes de realizar cualquier llamada a los servicios en la nube para ahorrar ancho de banda y procesamiento.			
Evidencias:	Registro de consola de Playwright: [chromium] > tests\e2e\01-login.spec.js:10:3 > Flujo de Autenticación - ParkScan > Validación de campos obligatorios (PASSED).			
Datos de Prueba:				
Inputs vacíos.				
Pre-condiciones para ejecución:				
El servidor web local de frontend debe estar activo en http://localhost:5173. Estar posicionado en la URL /admin/login.				
Post-condiciones:				
El formulario no debe enviarse ni recargar la página. El foco visual se mantiene en el primer campo vacío.				
Ejecución:				
Paso	Condición	Valor	Resultado Esperado	Resultado Obtenido
1	Navegar a la sección de login administrativo	URL /admin/login	La página de acceso carga con el formulario en blanco.	Pantalla de login renderizada de forma exitosa.
2	Presionar el botón "Iniciar Sesión"	Clic en button[type="submit"]	El formulario no se envía. El input de usuario se marca como	El navegador detiene el envío. Playwright comprueba la propiedad



			:invalid debido al atributo HTML5 required.	validity.va lueMissing == true en el input.
Paso	Condición	Valor	Resultado Esperado	Resultado Obtenido
Criterios de Aprobación:La funcionalidad está creada por vista correctamente y el navegador restringe el envío si los campos obligatorios están ausentes.				
Decisión de Aprobación:	Aprobó: X		Falló:	
Estado del Caso:	● Ejecutado y Aprobado exitosamente.			
Fecha de Aprobación/No Aprobación: 14/05/2026				

CP-PROYECTO-002

## Plantilla de Caso de Prueba

<b>ID:</b> CP-p-002	CP--002
<b>Título:</b>	Login Inválido (Rechazo de credenciales incorrectas en UI)

ID Redmine del caso:	N/A
Defectos asociados (IDs Redmine):	Ninguno
Prioridad: (Dada por el dev o PO)	Alta
Severidad:	Crítica
Estado de defectos:	N/A
Versión:	v1.2.0 (Etapa 4)
Sistema:	ParkScan - Sistema de Playa de Estacionamiento
Módulo:	Autenticación de Administradores
Escenario:	Un usuario ingresa credenciales incorrectas o inexistentes. El sistema debe responder de manera amigable informando que las credenciales no son válidas.
Nivel Prueba:	Sistema
Tipo Prueba:	Funcional (Seguridad y Robustez)
Ambiente de Prueba:	Localhost / Servidor de Desarrollo (Vite <code>http://localhost:5173</code> )
Autor:	QA(Maximo Muñoz)
Responsable ejecución:	Playwright E2E Runner
Fecha Creación:	14/05/2026
Fecha Ejecución:	14/05/2026
Requerimientos / reglas de negocio asociadas	RQ-03: Solo los usuarios registrados y habilitados en la base de datos Supabase bajo el rol correspondiente pueden acceder al panel administrativo.   RQ-04: La interfaz de usuario debe notificar explícitamente el rechazo de credenciales mediante un contenedor visual destacado sin revelar detalles técnicos sensibles de la base de datos.
Evidencias:	<pre>[chromium] &gt; tests\e2e\01-login.spec.js:22:3 &gt; Flujo de Autenticación - ParkScan</pre>

> Login Inválido - Credenciales incorrectas (PASSED).				
<b>Datos de Prueba:</b>				
Usuario: admin_falso Contraseña: 123456				
<b>Pre-condiciones para ejecución:</b>				
El servidor web local de frontend debe estar activo en http://localhost:5173. Estar posicionado en la URL /admin/login.				
<b>Post-condiciones:</b>				
El usuario permanece en la misma página de inicio de sesión /admin/login. Se mantiene el formulario de login pero con los campos de credenciales reseteados o listos para volver a intentar, mostrando el banner de alerta en pantalla.				
<b>Ejecución:</b>				
Paso	Condición	Valor	Resultado Esperado	Resultado Obtenido
1	Navegar a la pantalla de login de administración	URL /admin/login	Carga de formulario de acceso.	Pantalla de login renderizada exitosamente.
2	Introducir datos de acceso incorrectos	Input usuario = admin_falso  Input password = 123456	El texto ingresado es legible en el input de usuario y enmascarado con viñetas en el password.	Datos ingresados correctamente en los campos correspondientes .
3	Enviar el formulario haciendo clic en el botón de acceso	Clic en button[type="submit"]	Petición enviada a la API de Supabase, que responde con código de rechazo. El frontend lee el fallo y renderiza el	Se envía la petición. Supabase rechaza los datos. La UI despliega inmediatamente el banner .text-red-4

			texto de alerta "Credenciales incorrectas".	00 conteniendo el texto esperado.
Criterios de Aprobación: La UI detecta correctamente el rechazo de credenciales de la base de datos y dibuja el texto de error rojo explicativo en pantalla sin permitir el paso al Dashboard.				
Decisión de Aprobación:	Aprobó: X		Falló:	
Estado del Caso:	Ejecutado y Aprobado exitosamente.			
Fecha de Aprobación/No Aprobación: 14/05/2026				

CP-PROYECTO-001

## Plantilla de Caso de Prueba

<b>ID:</b> CP-03	CP-03
<b>Título:</b>	Login Válido (Inicio de sesión exitoso y redirección)
<b>ID Redmine del caso:</b>	N/A
<b>Defectos asociados (IDs Redmine):</b>	Ninguno
<b>Prioridad: (Dada por el dev o PO)</b>	Crítica
<b>Severidad:</b>	Bloqueante
<b>Estado de defectos:</b>	N/A
<b>Versión:</b>	v1.2.0 (Etapas 4)
<b>Sistema:</b>	ParkScan - Sistema de Playa de Estacionamiento
<b>Módulo:</b>	Autenticación de Administradores
<b>Escenario:</b>	Un administrador con credenciales válidas inicia sesión en el sistema. El sistema debe autorizar el acceso, almacenar los datos de la sesión localmente y redirigir inmediatamente a /admin/dashboard.

Nivel Prueba:	Sistema / Aceptación
Tipo Prueba:	Funcional (Flujo Principal)
Ambiente de Prueba:	Localhost / Mocks de Red en Playwright (page.route para Supabase REST)
Autor:	QA(Maximo Muñoz)
Responsable ejecución:	Playwright E2E Runner
Fecha Creación:	14/05/2026
Fecha Ejecución:	14/05/2026
Requerimientos / reglas de negocio asociadas	RQ-05: Ante credenciales válidas devueltas por la base de datos, el cliente debe persistir la información del usuario de manera que la sesión permanezca activa (uso de localStorage con clave parkscan_admin).   RQ-06: Una vez autenticado, el usuario de perfil administrador debe ser redirigido inmediatamente a la vista interna /admin/dashboard para realizar la gestión operativa.
Evidencias:	Registro de consola de Playwright: [chromium]  <pre>&gt; tests\e2e\01-login.spec.js:34:3 &gt; Flujo de Autenticación - ParkScan &gt; Login Válido - Redirección al Dashboard (PASSED).</pre>
<b>Datos de Prueba:</b>	
<b>Usuario:</b> admin   <b>Contraseña:</b> password_real   <b>Mock de API Supabase:</b> Devolver JSON [{ id: 1, usuario: 'admin', rol: 'admin', nombre: 'Admin Mock' }] con status HTTP 200 ante peticiones a <code>**/rest/v1/administradores*</code>.	
<b>Pre-condiciones para ejecución:</b>	
El servidor de desarrollo debe estar operativo en <code>http://localhost:5173</code>.   Mock de intercepción de red activo en Playwright para simular respuesta exitosa de la base de datos.   Estar posicionado en la URL <code>/admin/login</code>.	
<b>Post-condiciones:</b>	

El usuario ingresa a la URL interna `/admin/dashboard`.

La clave `parkscan_admin` queda seteada en el `localStorage` del navegador.

**Ejecución:**

Paso	Condición	Valor	Resultado Esperado	Resultado Obtenido
1	Cargar la pantalla de login con el mock de API preparado	URL <code>/admin/login</code> con interceptor activo	Pantalla renderizada lista para recibir credenciales.	Pantalla cargada e interceptor de red inyectado en segundo plano.
2	Ingresar las credenciales válidas del administrador	Input usuario = <code>admin</code> Input password = <code>password_real</code>	Datos cargados en los inputs de pantalla.	Campos rellenos con los datos correspondientes de prueba.
3	Presionar el botón "Iniciar Sesión" para procesar el acceso	Clic en <code>button[type="submit"]</code>	El frontend llama a la API. El mock responde exitosamente (200 OK). La aplicación guarda la sesión en <code>localStorage</code> y redirige al panel.	Se procesa la autenticación exitosa. Se escribe la clave de sesión en <code>localStorage</code> y ocurre la redirección automática.
4	Validar la redirección y almacenamiento final	Aserción sobre URL y almacenamiento	La URL final es <code>http://localhost:5173/admin/dashboard</code> y <code>localStorage</code> contiene el string <code>usuario: "admin"</code> .	URL redireccionada a <code>/admin/dashboard</code> . El token de sesión persiste correctamente con valor idéntico al mock.

Paso	Condición	Valor	Resultado Esperado	Resultado Obtenido
Criterios de Aprobación: El administrador se autentica correctamente ante datos válidos, su sesión queda persistida de forma local en localStorage y es enviado inmediatamente a su panel administrativo de control.				
Decisión de Aprobación:		Aprobó: X	Falló:	
Estado del Caso:		Ejecutado y Aprobado exitosamente.		
Fecha de Aprobación/No Aprobación:14/05/2026				

CP-PROYECTO-004

## Plantilla de Caso de Prueba

<b>ID:</b> CP-p-004	CP--004
<b>Título:</b>	Consumo exitoso de API Externa (Generación de link de pago Mercado Pago)
<b>ID Redmine del caso:</b>	N/A
<b>Defectos asociados (IDs Redmine):</b>	Ninguno
<b>Prioridad: (Dada por el dev o PO)</b>	Crítica
<b>Severidad:</b>	Crítica
<b>Estado de defectos:</b>	N/A

<b>Versión:</b>	v1.2.0 (Etapa 4)
<b>Sistema:</b>	ParkScan - Sistema de Playa de Estacionamiento
<b>Módulo:</b>	Portal de Clientes - Pagos con QR
<b>Escenario:</b>	Un cliente escanea/introduce su código de ticket de estacionamiento activo para pagar. El sistema consulta el ticket y consume la API de Mercado Pago mediante Supabase Edge Functions para generar un enlace de checkout.
<b>Nivel Prueba:</b>	Sistema / Integración de API
<b>Tipo Prueba:</b>	Funcional y de Integración
<b>Ambiente de Prueba:</b>	Localhost / Mocks de Red en Playwright (page.route para Supabase REST y Edge Functions)
<b>Autor:</b>	QA (Maximo Muñoz)
<b>Responsable ejecución:</b>	Playwright E2E Runner
<b>Fecha Creación:</b>	14/05/2026
<b>Fecha Ejecución:</b>	14/05/2026
<b>Requerimientos / reglas de negocio asociadas</b>	RQ-07: Al procesar el pago de un ticket activo, se debe calcular la tarifa de estacionamiento basada en el tiempo transcurrido (tarifa por hora del sector correspondiente).   RQ-08: El sistema debe consumir de forma asíncrona la Edge Function create-preference para interactuar de forma segura con la API externa de Mercado Pago y obtener la URL de pago real (init_point).   RQ-09: El frontend debe mostrar un botón interactivo y visible con el texto "Abrir en Mercado Pago (Web)" que redirija al cliente al checkout oficial utilizando el enlace provisto por la API.
<b>Evidencias:</b>	Registro de consola de Playwright: [chromium]  <pre>&gt; tests\e2e\02-pago-api.spec.js:25:3 &gt; Flujo de Pago e Integración API - ParkScan &gt; Consumo Exitoso de API - Generación de link de Mercado Pago (PASSED).</pre>



Datos de Prueba:				
PS-1234-5678   <b>Mock de API Supabase REST:</b> Devolver ticket activo con <code>id_ticket: 'test-uuid-123'</code> y sector <code>Planta Baja</code> al consultar por <code>codigo_qr</code>.   <b>Mock de Edge Function ('create-preference'):</b> Devolver JSON <code>{ id: 'pref-mock-999', init_point: 'https://www.mercadopago.com.ar/mock-checkout' }</code> con status HTTP 200.				
Pre-condiciones para ejecución:				
Servidor local activo en <code>http://localhost:5173</code>.   Estar posicionado en la URL <code>/pago</code>.   Mocks de API del ticket y pasarela de pago inyectados y listos en Playwright.				
Post-condiciones:				
Se visualiza en pantalla el componente del ticket con los datos del vehículo y el sector asignado.   Se renderiza de manera exitosa el botón interactivo de Mercado Pago con el atributo <code>href</code> seteado exactamente a <code>https://www.mercadopago.com.ar/mock-checkout</code>.				
Ejecución:				
Paso	Condición	Valor	Resultado Esperado	Resultado Obtenido
1	Cargar la pantalla de pago de clientes	URL <code>/pago</code> con interceptores activos	Pantalla cargada mostrando el input de búsqueda de ticket QR.	Vista del cliente renderizada correctamente con el formulario listo.
2	Introducir un ticket activo válido en el campo de texto	Código = <code>PS-1234-5678</code>	El código de ticket es ingresado en el input <code>PS-XXXX-XXXX</code> .	Input completado de manera exitosa.
3	Enviar la búsqueda	Disparo del evento	El sistema consulta el ticket y	El frontend detecta la confirmación.

	presionando la tecla "Enter"	<code>press('Enter')</code> en el input	automáticamente invoca la Edge Function de cobro. Se genera la preferencia en la pasarela.	Llama a las APIs interceptadas. Los mocks responden exitosamente.
4	Validar la aparición en pantalla del enlace de pago	Aserción de visibilidad y atributos del enlace	Se renderiza el botón "Abrir en Mercado Pago (Web)". El enlace apunta a <code>https://www.mercadopago.com.ar/mock-checkout</code> .	Botón "Abrir en Mercado Pago" visible al instante. Atributo <code>href</code> validado correctamente en la aserción de Playwright.
Criterios de Aprobación: La aplicación consume correctamente el endpoint de pagos, parsea la respuesta JSON y enlaza adecuadamente al cliente con el flujo de Mercado Pago simulado mediante el <code>init_point</code> correspondiente.				
Decisión de Aprobación:		Aprobó: X	Falló:	
Estado del Caso:		● Ejecutado y Aprobado exitosamente.		
Fecha de Aprobación/No Aprobación: 14/05/2026				

CP-PROYECTO-001

## Plantilla de Caso de Prueba

<b>ID:</b> CP-05	<b>CP-05</b>
<b>Título:</b>	Comportamiento esperado ante errores de la API o ausencia de respuesta
<b>ID Redmine del caso:</b>	N/A

Defectos asociados (IDs Redmine):	<i>Ninguno</i>
Prioridad: (Dada por el dev o PO)	Alta
Severidad:	Crítica
Estado de defectos:	<i>N/A</i>
Versión:	v1.2.0 (Etapa 4)
Sistema:	ParkScan - Sistema de Playa de Estacionamiento
Módulo:	Portal de Clientes - Pagos con QR
Escenario:	Un cliente intenta generar su enlace de pago para un ticket válido pero la API externa de Mercado Pago está caída o devuelve un error interno (500). El sistema debe comportarse de forma resiliente, controlando el error en código sin crasheos y alertando amigablemente de la desconexión.
Nivel Prueba:	Sistema / Resiliencia y Tolerancia a Fallos
Tipo Prueba:	Funcional / Estabilidad
Ambiente de Prueba:	Localhost / Intercepción de red forzando error HTTP 500
Autor:	QA (Maximo Muñoz)
Responsable ejecución:	Playwright E2E Runner
Fecha Creación:	14/05/2026
Fecha Ejecución:	14/05/2026
Requerimientos / reglas de negocio asociadas	RQ-10: La aplicación web cliente no debe quedar congelada o crasheada ("pantalla en blanco") ante errores de APIs de terceros.   RQ-11: Cualquier error de red o de código de estado (5xx, 4xx, timeouts) en los servicios externos debe ser atrapado mediante estructuras <code>try/catch</code>.   RQ-12: Se debe informar al usuario final mediante un modal nativo o componente alert que el servicio de pago no responde en este momento, sugiriendo intentar más tarde.
Evidencias:	Registro de consola de Playwright: <code>[chromium]</code>  <pre>&gt; tests\e2e\02-pago-api.spec.js:54:3</pre>

> Flujo de Pago e Integración API - ParkScan > Comportamiento ante Error de API - Caída de Mercado Pago (PASSED).				
<b>Datos de Prueba:</b>				
<b>Código Ticket:</b> PS-1234-5678 <b>Mock de API Supabase REST:</b> Devolver ticket activo con <code>id_ticket: 'test-uuid-123'</code> al consultar. <b>Mock de Edge Function ('create-preference'):</b> Forzar respuesta de error con status HTTP 500 y JSON <code>{ error: 'Mercado Pago no responde temporalmente' }</code> .				
<b>Pre-condiciones para ejecución:</b>				
Servidor local activo en <code>http://localhost:5173</code> . Estar en la página <code>/pago</code> . Mocks configurados en Playwright, forzando la respuesta de la pasarela a código de error 500.				
<b>Post-condiciones:</b>				
● El frontend atrapa la excepción de la Edge Function en su bloque <code>catch</code>.     ● La interfaz del cliente no crashea, permaneciendo interactiva.     ● Se dispara el diálogo alert en pantalla mostrando el texto específico de caída.				
<b>Ejecución:</b>				
Paso	Condición	Valor	Resultado Esperado	Resultado Obtenido
1	Cargar la pantalla de pago de clientes con el mock de caída activo	URL <code>/pago</code> con interceptor configurado a error 500	Pantalla de pago renderizada de forma normal esperando input.	Carga normal del portal de pago de clientes.
2	Ingresar código de ticket activo y disparar la búsqueda	Código = <code>PS-1234-5678</code> y presionar "Enter"	El sistema busca el ticket e intenta llamar a la Edge Function, la	Se envía el ticket. Se detecta el fallo del servicio mock con status HTTP 500.

			cual falla devolviendo un status 500.	
3	Verificar comportamient o de resiliencia del frontend	Aserción sobre el evento <code>dialog</code> del navegador	La aplicación atrapa la respuesta fallida de la API externa, no se bloquea ni se rompe, y despliega una ventana de alerta informando al cliente.	La alerta nativa se dispara con éxito. Playwright captura el mensaje del diálogo: "Mercado Pago no responde temporalmente".
Criterios de Aprobación: La aplicación maneja de forma segura las excepciones de la API externa de Mercado Pago mediante bloque <code>catch</code> , previniendo congelamientos e informando debidamente de la incidencia técnica al usuario.				
Decisión de Aprobación:		Aprobó: X	Falló:	
Estado del Caso:		Ejecutado y Aprobado exitosamente.		
Fecha de Aprobación/No Aprobación: 14/05/2026				

### 3. Evidencia de Ejecución Realizada (Playwright)

Los 5 casos de prueba anteriores no fueron meramente diseñados teóricamente; se tradujeron en suites de pruebas de integración y extremo a extremo (E2E) robustas e independientes escritas en JavaScript sobre **Playwright Test Runner**.

A continuación, se detalla la evidencia exacta recopilada al correr los tests funcionales de regresión en el ambiente local de desarrollo.

#### Captura de Ejecución en Terminal (Test Runner Run)

```
PS C:\Users\Admin\Desktop\cosas\ParkScan> npx playwright test

Running 5 tests using 2 workers

[1/5] [chromium] > tests\e2e\01-login.spec.js:22:3 > Flujo de
Autenticación - ParkScan > Login Inválido - Credenciales incorrectas
```

```
[2/5] [chromium] > tests\e2e\01-login.spec.js:10:3 > Flujo de Autenticación - ParkScan > Validación de campos obligatorios
[3/5] [chromium] > tests\e2e\01-login.spec.js:34:3 > Flujo de Autenticación - ParkScan > Login Válido - Redirección al Dashboard
[4/5] [chromium] > tests\e2e\02-pago-api.spec.js:25:3 > Flujo de Pago e Integración API - ParkScan > Consumo Exitoso de API - Generación de link de Mercado Pago
[5/5] [chromium] > tests\e2e\02-pago-api.spec.js:54:3 > Flujo de Pago e Integración API - ParkScan > Comportamiento ante Error de API - Caída de Mercado Pago

5 passed (15.2s)
```

**Tasa de éxito:** 100% (5/5 tests pasados).

**Tiempo total de ejecución:** 15.2 segundos con 2 hilos paralelos.

**Navegadores cubiertos:** Chromium Headless Shell (v1223).

**Reporte HTML generado:** Ubicado localmente en <playwright-report/index.html>.

## 4. Análisis de Incidentes Encontrados y Soluciones de Robustez

Durante la fase de diseño técnico y automatización de las pruebas para la Etapa 4, se detectaron incidencias arquitectónicas clave del lado del frontend que fueron solventadas exitosamente:

### 1. El Incidente de la Respuesta Vacía en Supabase (Manejo de Error PGRST116)

- **Problema encontrado:** Durante la validación del Login Inválido (CP-02), al mockear la respuesta de la base de datos de Supabase simulando que no existía el administrador mediante un array de datos vacío (`[]`), el cliente oficial de `@supabase/supabase-js` arrojaba una excepción nativa con el código de error `PGRST116`. Esto causaba una interrupción en el flujo de Javascript si no era interceptado correctamente por un bloque de control de errores.
- **Solución implementada:** Se reestructuró la función de autenticación en `src/components/Login.jsx` envolviendo la consulta en un bloque `try-catch` robusto. Se configuró para que si la base de datos devuelve una respuesta vacía o un código de error, el catch atrape la excepción y asigne limpiamente la variable de estado `error` a `"Credenciales incorrectas"`. Esto permitió pasar la prueba sin crasheos y notificar limpiamente al usuario.

### 2. Prevención de Bypass de Pagos (Validación Cifrada contra Webhooks de Mercado Pago)

- **Problema/Vulnerabilidad analizada:** En la integración de la pasarela de pagos (CP-04), existía el riesgo de seguridad común de *Bypass de Pago*, donde un cliente malintencionado podría enviar solicitudes falsificadas de webhooks a nuestro backend para marcar un ticket como "Pagado" sin haber completado la transacción real en Mercado Pago.
- **Solución implementada:** Se comprobó e implementó en la Edge Function de webhook (mp-webhook) que el backend **no confía** en los estados ni montos enviados por el cliente o los parámetros visibles del webhook. En su lugar, el sistema toma únicamente el identificador cifrado de transacción (paymentId) y realiza una llamada del lado del servidor directa y segura a las APIs protegidas de Mercado Pago para validar la autenticidad y el estado real ("approved") del pago antes de actualizar el ticket en Supabase. Esto anula por completo el bypass y garantiza la integridad financiera de la playa de estacionamiento.

## Firmas de Aprobación de Calidad

Este reporte constituye el entregable oficial de testing para la liberación de las capacidades de seguridad, autenticación e integración externa del sistema **ParkScan**.

- **Autor de Pruebas:** *Equipo de QA*
- **Responsable de Aprobación Técnica:** *Playwright Automation Suite*
- **Estado Técnico General:** Apto para Producción (100% de Pruebas Aprobadas)